Multivariate Plot

Scatter Plots, Overplotting, Bubble Charts

Xu Shirong

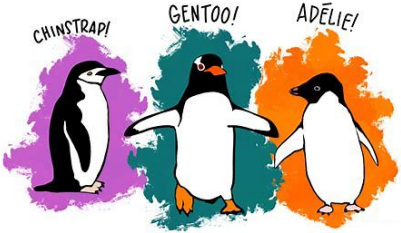
School of Economics, Xiamen University

2026

Penguins Dataset Overview

- Categorical: species, island, sex
- Numeric: bill length, depth, flipper length, body mass
- Ordinal: year

Goal: Visualize distributions and differences by sex and species



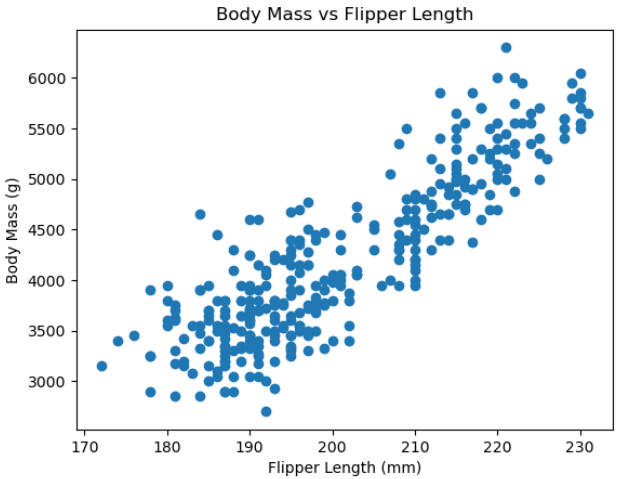
Basic Scatter (plt.scatter)

- It uses position on the x and y axes to show values for two variables.

```
import seaborn as sns
import matplotlib.pyplot as plt
penguins = sns.load_dataset("penguins").dropna()
plt.scatter(penguins["flipper_length_mm"], penguins["body_mass_g"])
plt.xlabel("Flipper Length (mm)")
plt.ylabel("Body Mass (g)")
plt.title("Body Mass vs Flipper Length")
plt.show()
```

A scatter plot illustrates the relationship between two variables, allowing us to observe how one changes with respect to the other. When interpreting a scatter plot, we typically focus on two key aspects: (1) the overall trend and (2) the slope of the relationship.

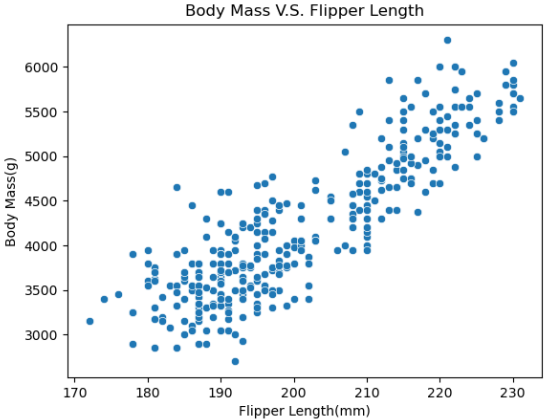
Basic Scatter (plt.scatter)



1. As Flipper Length increases, the body mass increases.
2. Their relationship is almost linear.

Basic Scatter (sns.scatterplot)

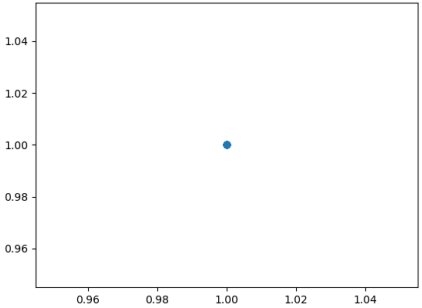
```
sns.scatterplot(data=penguins,x="flipper_length_mm",y="body_mass_g")
plt.title("Body Mass vs Flipper Length")
plt.show()
```



What is the main problem of Scatter Plot

- Sometimes, many points are stacked at the same location (see the following example).
- In a simple scatter plot, this results in many points overlapping—partially or completely—this is known as overplotting.

```
X, Y = np.ones(10), np.ones(10)
plt.scatter(X,Y)
```

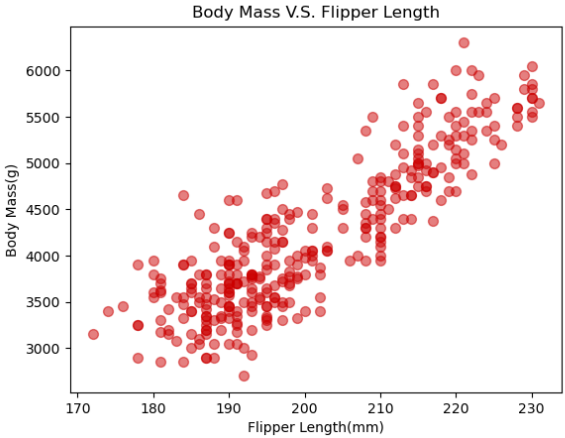


How to fix this problem?

1. Make points transparent.
2. Jitter points.

Partial Transparency

```
plt.scatter(penguins["flipper_length_mm"],penguins["body_mass_g"],alpha=0.5, color="
    #CC0000", s=50)
```

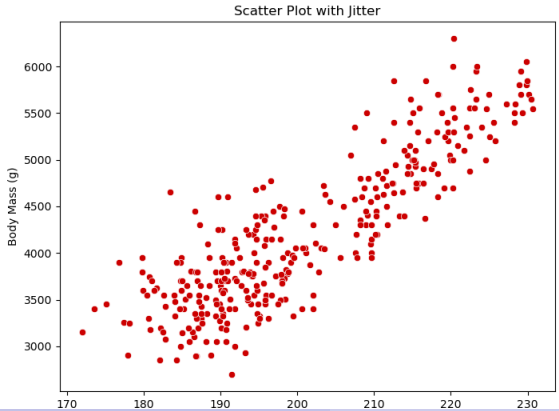


`alpha=0.5` makes points transparent

- Overlap points are darker.

Jitter

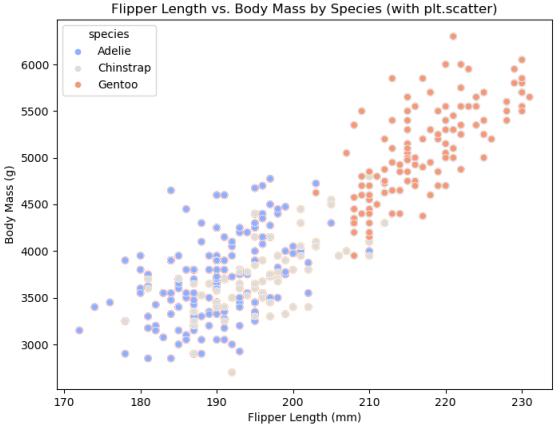
```
x_j = penguins["flipper_length_mm"] + np.random.normal(0, 0.5, len(penguins))
y_j = penguins["body_mass_g"] + np.random.normal(0, 1, len(penguins))
sns.scatterplot(x=x_j, y=y_j, color="#CC0000")
```



- Jittering makes points less likely to be overlapped.
- Destroy the raw data distribution

Hue by Species (Seaborn)

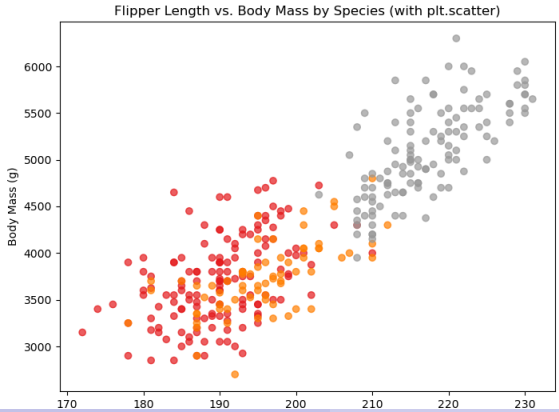
```
sns.scatterplot(data=penguins,x="flipper_length_mm",y="body_mass_g",hue="species",
    palette="coolwarm")
```



- Different colors correspond to different species.
- Three categories have the same pattern.

Hue by Species (Matplotlib)

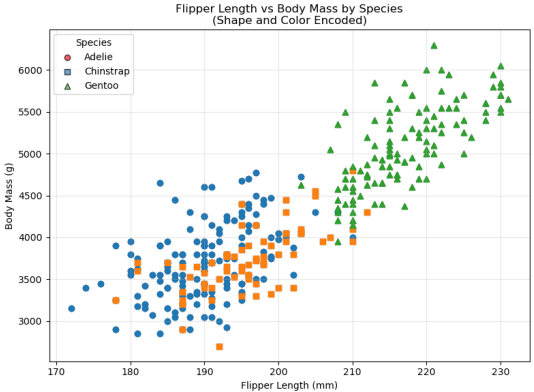
```
colors = penguins["species"].map({"Adelie":0,"Chinstrap":1,"Gentoo":2})
plt.scatter(penguins["flipper_length_mm"],penguins["body_mass_g"], c=colors, cmap="
    Set1")
```



- The key difference between Matplotlib and seaborn is we have to define colors according to group indexes.
- Key problem in visualization?

Shape + Color Encoding

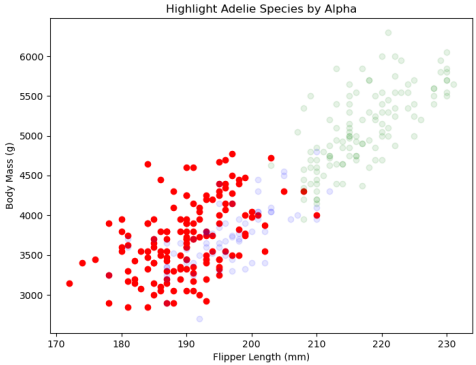
```
for species, marker in {"Adelie":"o","Chinstrap":"s","Gentoo":"^"}.items():
  g = penguins[penguins["species"]==species]
  plt.scatter(g["flipper_length_mm"], g["body_mass_g"],
              label=species, marker=marker)
```



- We can assign different shapes to different groups to enhance their difference.
- What if we wanna highlight a specific group, say Adelie.

Highlight One Group

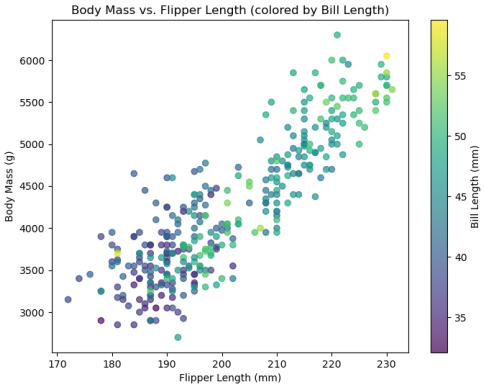
```
alpha_map = {"Adelie":1.0,"Chinstrap":0.1,"Gentoo":0.1}
for s,g in penguins.groupby("species"):
  plt.scatter(g["flipper_length_mm"], g["body_mass_g"], alpha=alpha_map[s])
```



- We can set a greater alpha value to those groups we want to highlight.
- Can we add a new dimension of data (Bill Length (mm)) to scatter plot? If yes, what change we can impose on the scatter plot?

Continuous Color Gradient

```
plt.scatter(penguins["flipper_length_mm"],penguins["body_mass_g"],c=penguins["
    bill_length_mm"], alpha=0.7)
plt.colorbar(label="Bill Length (mm)")
```



- We can use the darkness (intensity) of the points as an additional dimension to represent bill length.
- One more dimension? How to introduce a new dimension.

Summary of Scatter Plot

- **x, y**: Coordinates of data points (array-like or Series)
- **s**: Size of markers (scalar or array, default = 20)
- **c**: Color of markers (single color or array for colormap)
- **cmap**: Colormap used when c is numeric
- **alpha**: Transparency (0 to 1)
- **marker**: Marker style (`"o"`, `"x"`, `"s"`, `"`, `"D"`, ...)

- `edgecolors: Color of marker edges (e.g., "black")`
- `linewidths: Width of marker edge lines`
- `label: Label for legend`
- `zorder: Drawing order (higher values appear on top)`

Overplotting in Scatter Plots

Overplotting occurs when many points in a scatter plot overlap, which is common for large datasets or repeated values. This can obscure underlying patterns in the data.

Strategies to reduce overplotting:

- **Partial transparency** (`alpha`)
- **Add jitter** (small random noise)
- **Use color** to differentiate groups

Strategy 4: Rug Plots

Rug plots add small tick marks along the axes to show the distribution of individual data points.

- Show marginal distributions of data
- Helpful for visualizing density
- Often combined with scatter plots

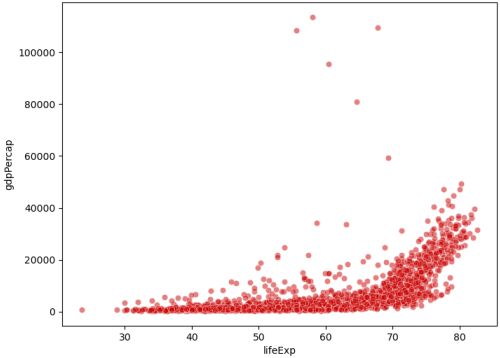
Gapminder Raw Dataset Overview

- **Source**: Gapminder (Hans Rosling team)
- **Coverage**: 142+ countries and regions
- **Variables**:
  - `country`: country name
  - `year`: observation year
  - `pop`: population size
  - `continent`: continent
  - `lifeExp`: life expectancy (years)
  - `gdpPercap`: GDP per capita

	country	year	pop	continent	lifeExp	gdpPercap
0	Afghanistan	1952	8425333.0	Asia	28.801	779.445314
1	Afghanistan	1957	9240934.0	Asia	30.332	820.853030
2	Afghanistan	1962	10267083.0	Asia	31.997	853.100710
3	Afghanistan	1967	11537966.0	Asia	34.020	836.197138
4	Afghanistan	1972	13079460.0	Asia	36.088	739.981106
5	Afghanistan	1977	14880372.0	Asia	38.438	786.113360
6	Afghanistan	1982	12881816.0	Asia	39.854	978.011439
7	Afghanistan	1987	13867957.0	Asia	40.822	852.395945
8	Afghanistan	1992	16317921.0	Asia	41.674	649.341395
9	Afghanistan	1997	22227415.0	Asia	41.763	635.341351

Gapminder Raw

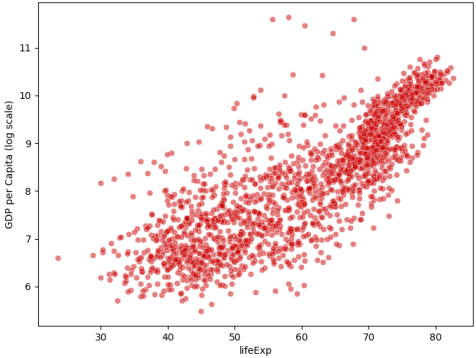
```
url = "https://raw.githubusercontent.com/resbaz/r-novice-gapminder/gh-pages/data/
    gapminder-FiveYearData.csv"
gapminder = pd.read_csv(url)
sns.scatterplot(data=gapminder,x="lifeExp", y="gdpPercap", alpha=0.5)
```



- In this example, the previous three methods may not work well since the overplotting problem is severe.
- In this example, where points are clustered around a specific area but there are many outliers, what approaches can we take?

Gapminder Log

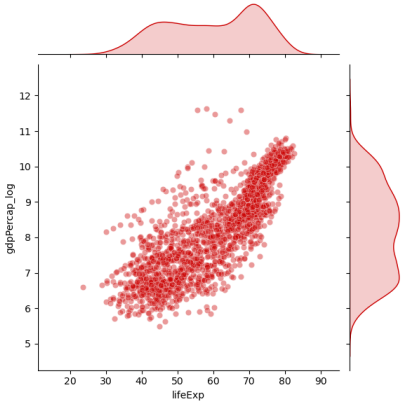
```
gapminder["gdpPercap_log"] = np.log(gapminder["gdpPercap"])
sns.scatterplot(data=gapminder,
  x="lifeExp", y="gdpPercap_log", alpha=0.5)
```



- A simple way is to implement data transformation such that points are separated?

Joint + KDE

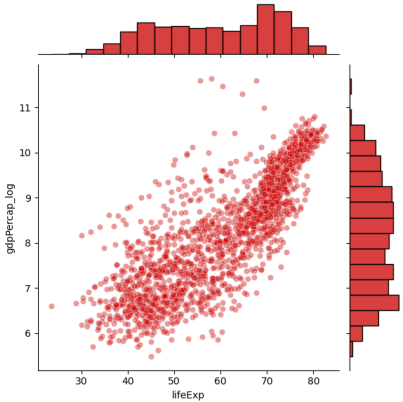
```
import seaborn as sns
g = sns.JointGrid(data=gapminder, x="lifeExp", y="gdpPercap_log")
g.plot_joint(sns.scatterplot, color="#CC0000", alpha=0.4)
g.plot_marginals(sns.kdeplot, fill=True, color="#CC0000", alpha=0.2)
```



- Adding a density plot to the scatter plot highlights areas where points are highly concentrated.
- Similarly, we can use other distribution plots to achieve the same result.

Joint + Hist

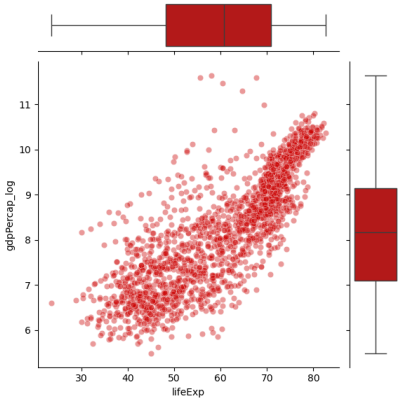
```
g = sns.JointGrid(data=gapminder, x="lifeExp", y="gdpPercap_log")
g.plot_joint(sns.scatterplot, color="#CC0000", alpha=0.4)
g.plot_marginals(sns.histplot, fill=True, color="#CC0000")
plt.ylabel("GDP per Capita (log scale)")
```



- Clearly, the tallest column corresponds to the region with the highest degree of overplotting.

Joint + Box

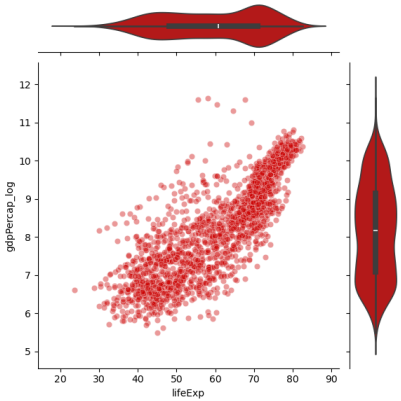
```
g = sns.JointGrid(data=gapminder, x="lifeExp", y="gdpPercap_log")
g.plot_joint(sns.scatterplot, color="#CC0000", alpha=0.4)
g.plot_marginals(sns.boxplot, fill=True, color="#CC0000")
plt.ylabel("GDP per Capita (log scale)")
```



- the overlap of two boxes represent the region with the highest degree of overplotting.

Joint + Violin

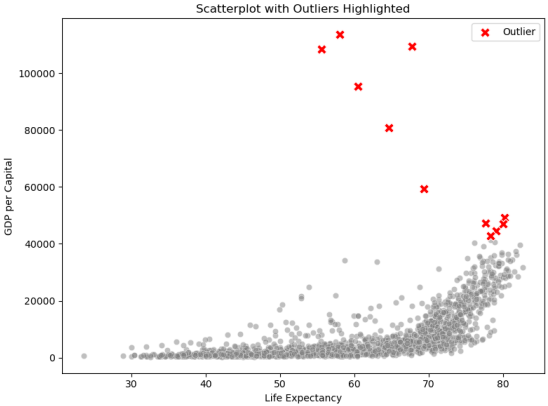
```
g = sns.JointGrid(data=gapminder, x="lifeExp", y="gdpPercap_log")
g.plot_joint(sns.scatterplot, color="#CC0000", alpha=0.4)
g.plot_marginals(sns.violinplot, fill=True, color="#CC0000")
plt.ylabel("GDP per Capita (log scale)")
```



- Similar interpretation as previous slide.

Outliers Highlighted

```
The codes here is a practice.
```



- How to understand outliers?

2D Histogram

```
sns.histplot(data=gapminder,x="lifeExp",y="gdpPercap_log",bins=50,cmap="Reds")
```

Advantages

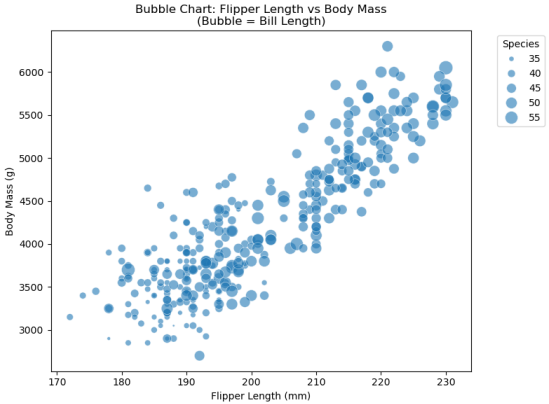
- Reveal density patterns and clusters in large datasets
- Reduce overplotting compared to scatter plots
- Easy to interpret overall trends and correlations

Disadvantages

- Sensitive to bin size and binning strategy
- Fine details and outliers may be hidden
- Color scale choice can strongly affect perception

Bubble Chart (Penguins)

```
sns.scatterplot(data=penguins,x="flipper_length_mm",y="body_mass_g",size="
    bill_length_mm",  sizes=(5, 200),  palette="Set2",alpha=0.6
)
```

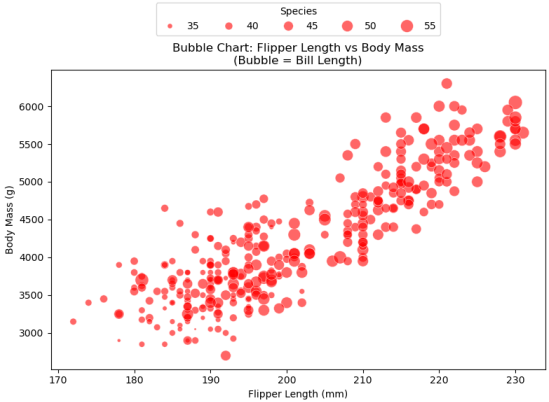


Advantages

- Encode three variables in a single plot (x, y, and bubble size)
- Intuitive visual representation of magnitude and scale
- Effective for showing relationships and trade-offs between variables
- Engaging and memorable for storytelling and presentations

Bubble Chart (Penguins)

```
sns.scatterplot(data=penguins,x="flipper_length_mm",y="body_mass_g",size="
    bill_length_mm",  sizes=(5, 200), alpha=0.6,color="red"
)
```



Advantages

- Encode three variables in a single plot (x, y, and bubble size)
- Intuitive visual representation of magnitude and scale
- Effective for showing relationships and trade-offs between variables
- Engaging and memorable for storytelling and presentations

Use Bubble Charts with Caution

- Bubble charts use **two different aesthetic scales** (position and size) to represent the same type of variable — **quantitative variables**.
- It is difficult to visually judge the **strength of association** between variables.
- Encoding value differences using **bubble size** is harder to interpret than using position.
- When data differences are small, **bubble size differences** become minimal and hard to detect.
- It is often difficult to **match bubble size differences** to the actual numerical differences in values.